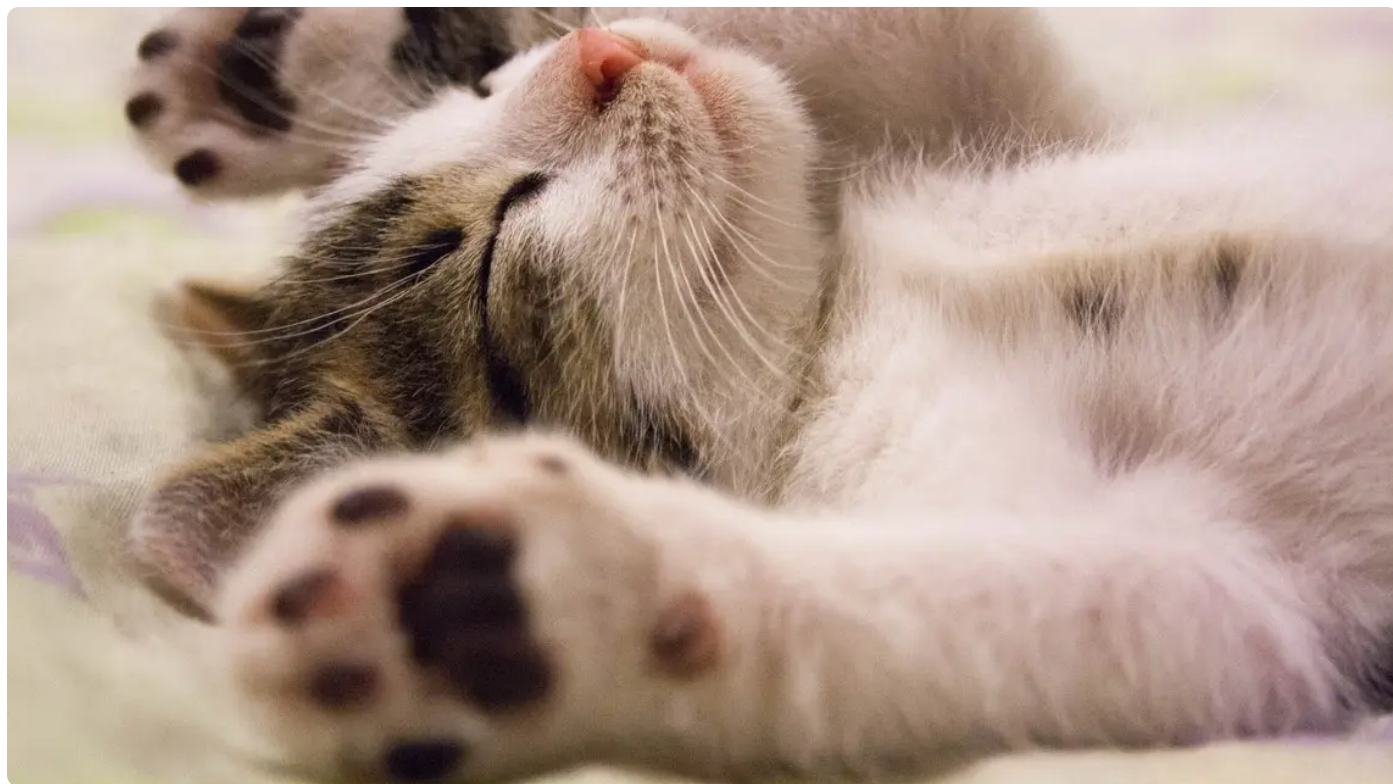


03 | Style：关于样式你需要知道的三件事

2022-4-1 蒋宏伟



你好，我是蒋宏伟。

上一讲我们说到，搭建页面的第一步是搭建静态页面，拿到设计稿后要从上往下拆成组件，再从下往上把组件进行实现。

但组件只是页面的架子。如果你不使用任何样式，组件只能遵循默认的布局规则、默认字号颜色，铺在屏幕上，看起来就像调试的 log 信息一样，也没有什么体验可言。

俗话说人靠衣装、佛靠金装，页面体验要好就离不开样式的帮助。大家对 App 的第一印象，就是对页面样式的第一印象。虽说样式设计上是由设计师负责，但最终落地还得靠代码。如何把设计师给的设计稿在不同大小的机型上还原实现，通过验收，是工作中实实在在要面对的考验。

还原设计稿还只是最基本的要求，作为开发者，你还得要关心开发成本、可维护性、布局性能等事情。比如，有哪些样式库可以节约开发成本？代码量大了需求有变动，样式怎么改起来更方便？React Native 的布局性能究竟怎样，多层嵌套的复杂布局会不会导致性能问题？

所以今天，围绕着上面这些话题，我和你一起聊聊，关于样式你需要知道的三件事：

React Native 组件样式都有哪些？

React Native 的 Flex 布局有哪些特点？

React Native 样式代码如何管理？

组件样式 = 通用样式 + “私有”样式

我们先来说说，React Native 组件样式都有哪些。

还原设计稿离不开样式的支持，样式决定了组件在屏幕中的样子。大部分 React Native 提供的框架组件都有样式属性，也就是 style 属性。比如，你要改变文字的颜色，就需要给 Text 组件的 style 属性传一个 {color: 'red'} 对象。如果要设置文字一个圆角边框，那就要稍微复杂一点了，需要三个样式值：边框颜色 borderColor、边框宽度 borderWidth、边框半径 borderRadius，比如这段示例代码：

```
// 文字颜色
<Text style={{color:'red'}}>
// 圆角边框
<Text style={{borderColor:'green', borderWidth: 1, borderRadius: 5}}>
```

不过，不同组件的支持的样式可能会有些不同。比如，上面这段代码中，文字颜色 color 只有 Text 和 TextInput 组件有，图片组件 Image 没有文字也不需要 color 样式。而边框样式 border*（比如 borderColor、borderWidth、borderRadius 等等），容器组件 View、文字组件 Text、图片组件 Image 都有。

那我们怎么知道哪个组件都有哪些样式？要死记硬背吗？当然不用。

一方面，通过 TypeScript 声明文件，编辑器会提醒你某个组件都有哪些样式。另一方面，React Native 的组件样式是有规则的，你只需要把那些高频样式用会就可以了，其他低频样式，等要用到的时候再翻文档也不迟。

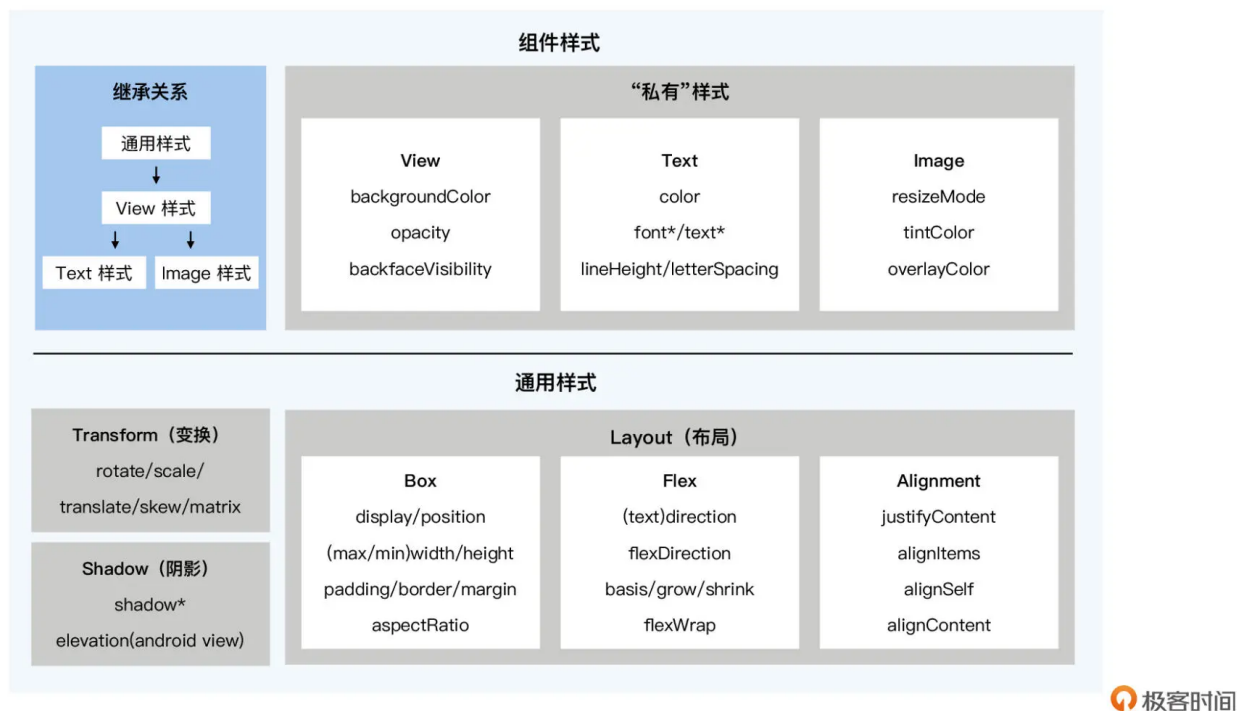
组件样式是有继承关系的，可以分为三层：

第一层是通用样式；

第二层是 View 组件样式；

第三层是 Text、Image 等其他组件样式。

我把组件样式的三层继承规则整理成了一张图片，相信你看完之后会有更深刻的理解：



通用样式包括布局 Layout、变换 Transform 和阴影 Shadow。容器组件要不要展示归布局 Layout 管，位置确定后要往左边挪点还是旋转个角度归变换 Transform 管，要立体感要加个阴影归 Shadow 管。

View 组件样式继承了所有通用样式，包括布局 Layout、变换 Transform、阴影 Shadow，除此之外，还有自己的“私有”样式，比如背景颜色 backgroundColor、透明度 opacity、背面可见 backfaceVisibility。另外，Android API 28 以下用的阴影属性 elevation 也是 View 的“私有”样式，为了记忆方便，你也可以将其归类到阴影 Shadow 上。

大部分组件，比如 Text、Image 组件，都继承了 View 组件样式。因此 View 组件的背景色 backgroundColor、Android 低版本阴影 elevation 等“私有”样式，其实也可以算作通用样式。

但 Text 组件、Image 组件的“私有”样式，就不能相互通用了。文字颜色 color、字体大小 fontSize、文字行高 lineHeight，这些是文字组件独有的，图片组件就不能用。图片大小模型样式 resizeMode 是图片独有的，文字组件也不能用。

简而言之，组件样式 = 通用样式 + “私有”样式，View 组件样式可以算作通用样式，而 Text 和 Image 组件各有各的“私有”样式。

Flex：跨平台、高性能、易上手

在所有样式中，你用的最多一定是布局样式（Layout），而布局样式中大部分都是 Flex 相关的弹性布局。

React Native 在 2015 年诞生之初，就选择使用 Flex 作为默认的布局方式，到现在为止也仅仅只支持了 Flex 弹性盒子布局和 Absolute 绝对定位这两种布局方法。而 Flex 这种布局方式，也经受住了时间的考验，得到更多开发者的认同。

Flex 布局有三个特点：**跨平台、高性能、易上手**。

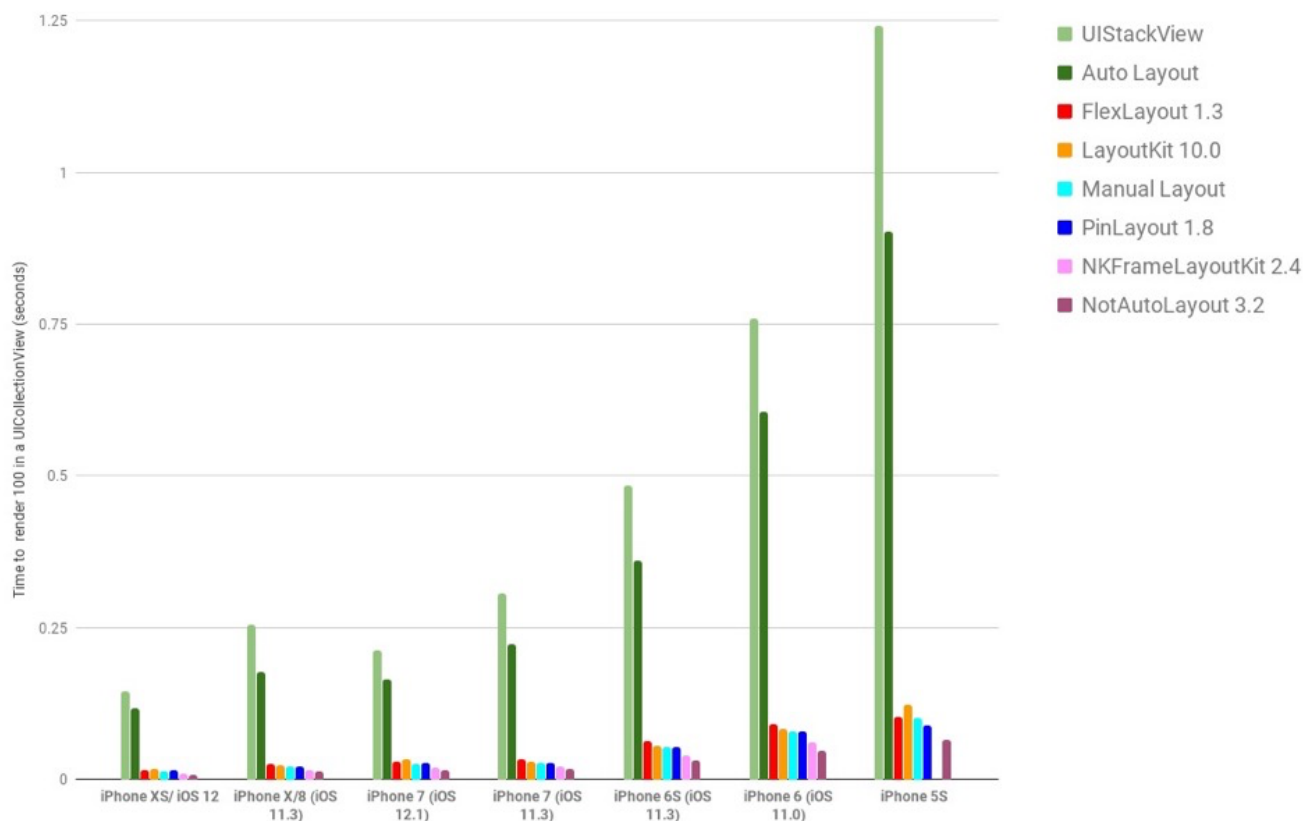
首先 Flex 布局是跨平台的，这里说的跨平台有两层含义。第一层含义是 Flex 布局并不是 React Native 所独有的，在 Web、Android、iOS 平台也都在用，Flex 布局知识的可迁移性很强。无论是前端开发还是客户端开发，你在你当前领域掌握的 Flex 知识，可以直接拿到 React Native 上用，反之亦然。

跨平台的第二层含义是，React Native 的布局引擎 Yoga 是 Android、iOS 通用的。你给组件写的 Flex 布局代码，最终都会被 Yoga 引擎计算为精确的坐标系，然后按照计算后的坐标系把组件渲染到屏幕上，这个布局计算在双端是一致的。

有些同学写代码的时候，可能一开始就担心，“这么写是不是会嵌套太深了，会不会引起布局性能问题？”，“设计师给的布局太复杂了，性能会不会不好啊？”。其实这些性能问题大可不必担心，正常写就行，Flex 布局用的 Yoga 引擎性能很好。

我这里放了一张布局引擎性能对比图，图片来源于 Github 开源仓库 [《Layout Framework Benchmark》](#)。核心代码贡献者 Luc Dion 是一位 iOS 开发工程师，他用 100 次 UICollectionView 布局耗时作为基准，横向对比了多款 iOS 布局引擎性能。其中就包括由苹果官方提供了 UIStackViews 和 Auto layout 布局引擎，还有使用 Yoga 实现 FlexLayout 布局引擎。

iPhone Comparison



在图中你可以看出，虽然 iPhone 每代的性能越来越好，100 次 UICollectionView 的布局耗时越来越少。但从框架性能角度看，使用 Yoga 实现的 FlexLayout 布局引擎比苹果官方提供了 UIViews 和 Auto layout 布局引擎，耗时减少了将近一个量级。

这样看来，React Native 中的 Flex 布局确实是挺好的，那上手难不难？不难，易学易用，上手就会。

前面我们也提到过，Flex 其实是一种通用的布局方式，它引入了弹性布局的概念，这个概念在各平台都是一样的。但在具体的写法上，各个平台可能会有一些差异。

我用最常见三种布局给你举些例子，它们包括从上往下排列布局、左图右文布局、文字居中布局。你可以感受一下，React Native 的 Flex 布局，和你在其他平台使用过的 Flex 布局有什么差异。

第一个例子，从上往下排列布局。

在同一个父容器中，放三个子容器 View，父容器不写任何的样式，子容器只给一个固定高度，三个子容器就是从上往下排列的。

这里强调一下，父容器 View 的默认样式是`{display: "flex",flexDirection: 'column'}`。也就是说，父容器是弹性盒子，且主轴是纵轴，子元素会沿着纵轴（主轴）方向排列，因此在父元素不写任何样式时，子元素是从上往下排列的。

示例代码如下：

```
<View>
  <View style={{height: 50, backgroundColor: 'powderblue'}} />
  <View style={{height: 50, backgroundColor: 'skyblue'}} />
  <View style={{height: 50, backgroundColor: 'steelblue'}} />
</View>
```

第二个例子，左图右文布局。

在同一个父容器中，放一个 Image 和一个 Text。为了让图片文字左右排列，我们需要给父容器设置布局样式`{flexDirection: 'row'}`。为了让图片不拉伸变形，我们需要给图片 Image 设置一个固定宽高。为了让文字将剩余宽度铺满，我们需要给文字 Text 设置 `{flex: 1}`。这时，父容器的主轴是横轴，子元素会沿着横轴（主轴）方向排列，整体布局是左图右文。具体的代码如下：

```
<View style={{flexDirection: 'row'}}>
  <Image
    style={{width: 100, height: 100}}
    source={{
      uri: 'https://placeimg.com/640/480/cats',
    }}
  />
  <Text style={{flex: 1,fontSize: 18}}>我是文字</Text>
</View>
```

第三个例子，文字居中布局。

曾经有一道经典的面试题，“父容器高度确定，使其子元素 Text 水平垂直方向居中”，不过自从有了 flex 后，这道题的难度降低了很多，问的频率也变低了。

我们通过 alignItems 和 justifyContent 的配合，很容易实现水平垂直方向的居中布局，示例代码如下：

```
<View
  style={{
    alignItems: 'center',
    justifyContent: 'center',
    // 高度确定
    height: 60,
    borderWidth: 1,
  }}>
  <Text
    style={{
      fontSize: 18,
      // 文字默认内边距，会导致垂直居中偏下
      includeFontPadding: false,
      // 文字默认基于基线对齐，会导致垂直居中偏下
      textAlignVertical: 'center',
    }}>
    我是文字1
  </Text>
</View>
```

在这段代码中，你只需要给父容器设置{justifyContent: 'center', alignItems: 'center'}，使子元素分别在主轴（纵轴）和副轴（横轴）方向居中就可以了。这里有个小细节，Android 文字默认会有内边距且基于基线对齐，这会导致文字垂直居中时偏下。**因此垂直居中时，最好把内边距关掉，并把文字放在中线而不是基线上。**

当然，文字水平垂直方向居中，除了 Flex 方案，还有行高方案，感兴趣的同学也可以自己研究一下，这里就不再介绍了。

讲完这三个例子后，你是否发现 React Native 与你所熟悉的其他平台，在 Flex 布局上的不同点了呢？你可以在心里对照一下，这样做能帮你学得更快。

StyleSheet：分离、复用、性能好

在前面的几个例子中，我们写样式用的都是内联的方式。内联样式就是直接在 JSX 的元素属性中写样式，这样写起来是很方便，但是却把 JSX 的元素结构和样式混在一起了。

既然样式属性可以内联，那事件属性也可以内联，甚至所有的属性都可以内联。而且现在 JSX 模板既要声明元素结构，又要写样式、事件、属性逻辑，整一个大杂烩。写起来是很爽，但维护起来就很“酸爽”了。

此外，内联样式还存在不能复用，性能损耗的问题。首先，即便两个文字组件的样式是一样的，内联样式也不能重复使用，必须在两个组件中各写一套。其次，每次执行自定义组件函数生成元素时，或实例化元素时，样式对象都要重复创建，这导致了性能损耗。你可以看看这段示例代码感受一下：

```
// 各种内联，导致 JSX 结构不清楚。
<View
  // 普通属性
  hitSlop={
    top: 10,
    bottom: 10,
    left: 0,
    right: 0
  }
  // 事件属性
  onLayout={() => {
    // 事件逻辑
  }}
  // 样式属性
  style={{
    alignItems: 'center',
    justifyContent: 'center',
    height: 60,
    borderWidth: 1,
  }}>
  <Text
    style={{
      fontSize: 18,
      includeFontPadding: false,
      textAlignVertical: 'center',
    }}>
    我是文字1
  </Text>
  <Text
    style={{
      fontSize: 18,
```



```
        includeFontPadding: false,  
        textAlignVertical: 'center',  
      }}>  
      我是文字2  
    </Text>  
  </View>
```

所以，我推荐你使用样式表 StyleSheet 来写样式，而不是内联的方式。使用样式表 StyleSheet 有三个好处：

- 元素结构和样式分离，可维护性更好；
- 样式对象可以复用，能减少重复代码；
- 样式对象只创建一次，也减少性能的损耗。

比如，面对上面这种大杂烩的代码，你可以试着把内联样式等属性抽离出来，没有了冗余的样式和属性，我们一眼就能看出原本的 JSX 结构：

```
// JSX 结构  
<View  
  hitSlop={hitSlop}  
  onLayout={handleLayout}  
  style={styles.container}>  
  <Text style={styles.texts}>我是文字1</Text>  
  <Text style={styles.texts}>我是文字2</Text>  
</View>
```

```
// 样式表  
const styles = StyleSheet.create({  
  container: {  
    alignItems: 'center',  
    justifyContent: 'center',  
    height: 60,  
    borderWidth: 1,  
  },  
  texts: {  
    fontSize: 18,  
    includeFontPadding: false,  
    textAlignVertical: 'center',  
  }  
});
```

你看，这是一个容器组件 View 嵌套了两个文字组件 Text。样式结构分离后，逻辑也更加清晰，维护起来也会容易很多。

而且，在这段代码中，两个 Text 组件使用了同一个样式对象 `styles.texts`，也实现了复用。样式对象在代码初始化时就创建好了，每次执行就不用再创建了，这样减少了性能损耗。

课程小结

我们前面说了，样式决定了页面的“颜值”，关于样式你需要知道这三件事：

1. 大部分框架提供的组件都有自己的样式属性 `style`，包括通用样式和“私有”样式。其中 View 组件样式可以看做通用样式，而 Text 组件、Image 组件各有各的“私有”样式；
2. 在所有样式中，最常用的是 Flex 布局，也是你的学习重点。React Native 的 Flex 布局和其他平台的 Flex 布局模型基本相同，如果你有过 Flex 的使用经验，只需结合示例掌握 React Native 中的那些不同点就能快学会；
3. 内联样式写 Demo 是没有问题的，但在实际的生产中我更加推荐你使用样式表 `StyleSheet` 来进行样式管理。

React Native 的样式大都是从 Web 中借鉴过来的，并且还进行了“CSS in JS”的改良，相信你学起来会非常快。

如果你问我学习样式还有什么技巧，那我会告诉你，无他，唯手熟尔。只要多多练习就能学好。学习样式不需要严格的推理逻辑，需要的只有勤加实践，当初我入门的时候，就是通过模仿国内电商的官网，把样式给打通关的，你也赶紧试试吧。

补充材料

样式学习材料：React Native 的样式其实很简单，所有的核心样式在的源码中只有 1 份声明文件 [StyleSheetTypes](#)。这一份声明文件对应的是官网的 6 篇文档：[View Style Props](#)、[Text Style Props](#)、[Image Style Props](#)、[Layout Props](#)、[Shadow Props](#)、[Transforms](#)。

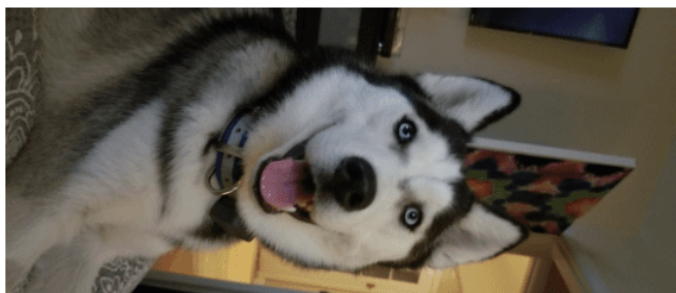
Flex 学习材料：Yoga 官网提供了 Flex 弹性盒子布局的在线试用应用 [Playground](#)，你可以动手把玩一下。React Native 官网也为你提供了沙盒环境的相关 [Demo](#)。

样式管理资料：今天只介绍了[样式表 StyleSheet](#)这种最基础的样式管理方案。业内主流的方案还有[带样式的组件 styledComponent](#) 和[样式简写方案 tailwind](#)，它们虽然是源自浏览器的 CSS 管理方案，但也可以在 React Native 中使用。在推特上也有关于样式管理方案的[讨论](#)，你可以看看大家的看法是什么。业务代码的样式管理没有银弹，选择适合你的就好了。

[今天的 Demo 在这里！](#)

作业

1. 请你使用 View、Text、Image 组件实现一个简易版的瀑布流布局，类似于京东、淘宝首页瀑布流列表，不要求能够无限滚动只要能够实现左右等宽、不等高的布局即可。



2. 如果你要给 Text 组件设置全局的默认样式，比如字体，你会怎么设置？

欢迎在评论区写下你的想法。我是蒋宏伟，咱们下节课见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 02 | Component：搭建静态页面的正确思路是什么？

下一篇 04 | State：如何让页面“动”起来？

精选留言 15